INFOESTE 2026 · ALÉM DO CÓDIGO

Guia Engenheiro

Do CSV sujo ao data warehouse: SQL, pandas, ETL e modelo dimensional

PARA QUEM

Quem vai construir o pipeline e o DW

INSTRUTORES

Lucas Araujo e [nome do parceiro]

VERSÃO

1.0 · outubro de 2026

Como usar este guia

Este guia acompanha os slides dos dois dias do curso. Ele foi escrito para quem vai **fazer o dado existir e ser confiável**: engenheiro de dados, engenheiro de analytics e quem quer começar na engenharia. A ideia é simples: ler, entender, testar no seu computador e só então chegar na aula.

Cada capítulo tem a mesma estrutura:

- **Em uma frase:** a ideia central, para memorizar.
- **Como funciona:** o conteúdo técnico, sem enrolação.
- **Na AgroMaq:** o mesmo conceito aplicado à fábrica do curso.
- **Teste você:** exercícios para fazer sozinho, com o resultado esperado para conferir.

> **Tempo total de estudo:** cerca de 10 a 12 horas, em quatro ou cinco sessões. O núcleo do guia é o laboratório dos capítulos 5, 7 e 9: é lá que você constrói, de ponta a ponta, o que vamos fazer na aula.

Mapa do guia

Capítulo	O que você aprende	Slides	Estudo
1. O cenário	AgroMaq, caminho do dado	Dia 1 · 2 a 7	20 min
2. As profissões	Onde o engenheiro entra	Dia 1 · 8 a 12	15 min
3. Ferramentas	SQL, janelas, pandas 3.0, SQLAlchemy	Dia 1 · 13 a 17	120 min
4. Ciclo de vida, OLTP × OLAP, ETL × ELT	Onde cada conceito se encaixa	Dia 1 · 18 a 20	30 min
5. Dado sujo	Seis defeitos, perfilamento em SQL e pandas	Dia 1 · 21, P1, P2, P4	120 min
6. Idempotência	Staging, transação, rodar duas vezes	Dia 1 · 22, P5	60 min
7. ETL: uma regra por defeito	Transformação, quarentena, gabarito, pico	Dia 1 · P3, P5, P6	120 min
8. Modelo dimensional	Fato, dimensão, granularidade, chaves	Dia 1 · 23 · Dia 2 · 3 e 4	60 min
9. Carga no DW	Dimensões, fatos, reconciliação	Dia 2 · 5, E1 a E3	120 min
10. Power BI, visto do DW	O que fica no Power Query e o que fica no DW	Dia 2 · 6 a 9	30 min
11. IA na engenharia	Delegar a escrita, não a conferência	Dia 1 · 24 a 26 · Dia 2 · 16	40 min
12. Carreira	Certificações e próximos passos	Dia 2 · 20, 21 e 23	20 min

O que você precisa instalar

Ferramenta	Para quê	Observação
MySQL 8 e MySQL Workbench	Rodar o banco e escrever SQL	Use o mesmo da aula
Python 3.11 ou mais novo	Rodar o pandas	Com `pandas` 3.0
`sqlalchemy` e `pymysql`	Ponte Python e MySQL	Opcional, para o Capítulo 3
Power BI Desktop e Oracle MySQL Connector/NET	Consumir o DW	Só para o Capítulo 10

> **Atenção.** Em máquinas de instituição, a instalação pode pedir permissão de administrador. Teste tudo **antes** da aula.

O laboratório de estudo

O laboratório tem um CSV com leituras de sensores reais do banco da AgroMaq, **com defeitos plantados em quantidades conhecidas**. Você vai perfilar, tratar, carregar e reconciliar. O passo a passo da instalação está no **Apêndice A**.

> **Importante.** O laboratório não é o CSV oficial da aula, e os nomes das tabelas dele podem diferir dos que usaremos em sala. Ele existe para você praticar as mesmas ideias. Os números dos "Teste você" valem para o laboratório, e cada exercício traz o **resultado esperado**. A solução completa fica no gabarito dos instrutores; **tente primeiro, sem olhar**.

Convenções

> **Dica.** Atalho ou boa prática que economiza tempo.

> **Atenção.** Armadilha comum. Quase todo mundo cai uma vez.

> **Conceito.** Definição para memorizar.

> **Teste você.** Exercício prático com resultado esperado.

1. O cenário: AgroMaq e o caminho do dado

Slides: Dia 1, 2 a 7 · Estudo: 30 minutos

> **Em uma frase.** Dado só vira decisão quando alguém o torna confiável.

A AgroMaq

Durante os dois dias, o cliente é a **AgroMaq**, uma fábrica de máquinas agrícolas com duas unidades:

Unidade	Cidade	O que fabrica
1	Presidente Prudente (SP)	Tratores e colhedoras
2	Três Lagoas (MS)	Plantadoras e pulverizadores

O banco operacional se chama `agromaq_industrial` e tem **38 tabelas**, **120 máquinas** e **316 sensores**. Cada sensor registra uma leitura por hora. No período completo do curso, de 01/01/2026 a 10/10/2026, isso dá cerca de **2,15 milhões de leituras** (316 sensores × 24 horas × 283 dias).

A hierarquia que você vai ver o tempo todo:

Nível	Exemplo
Unidade	Presidente Prudente
Linha de produção	Linha de Corte - Trator
Máquina	Máquina de Corte a Laser
Sensor	SEN00001 (Temperatura Motor)
Leitura	Um valor, em um horário

O caminho do dado até a decisão

BI (*Business Intelligence*) é o caminho que o dado percorre até virar uma decisão. São cinco etapas, e se uma falha, a decisão final sai errada:

Etapa	O que acontece	Exemplo na AgroMaq
1. Fontes	Onde o dado nasce	ERP, 316 sensores e planilhas de venda
2. Integração	O ETL tira o dado das fontes e o trata	Limpar leituras duplicadas
3. Armazenamento	Data warehouse com fatos e dimensões	Tabelas organizadas para análise

Etapa	O que acontece	Exemplo na AgroMaq
4. Análise	SQL e Power BI respondem às perguntas	Quantas leituras saíram da faixa?
5. Decisão	Alguém age	Parar a máquina ou ajustar a produção

> **Para o engenheiro.** Você é dono das etapas 2 e 3. Se uma delas falha, ninguém percebe até a etapa 5, quando alguém toma uma decisão errada. A pergunta que abre o curso é a que justifica o seu trabalho: **quem já viu um relatório com número errado?** Em geral, o erro nasceu antes da análise.

Dado, informação e conhecimento

Uma mesma leitura muda de valor conforme o contexto que ganha:

Camada	Exemplo (valor ilustrativo)	O que falta para subir
Dado	92 °C	Contexto: de qual sensor? Quando?
Informação	92 °C no sensor de temperatura do motor, às 14h, acima do limite da máquina	Histórico para explicar
Conhecimento	Esse rolamento aquece antes de falhar	Decisão
Decisão	Agendar a manutenção antes da próxima parada	

Três níveis de resposta

A mesma máquina permite perguntas cada vez mais difíceis, e cada degrau exige mais dado, **e dado de mais qualidade**:

Nível	Pergunta	Exemplo
Descritiva	O que aconteceu?	Quanto produzimos no mês?
Preditiva	O que vai acontecer?	Qual máquina vai falhar?
Prescritiva	O que fazer?	Qual manutenção agendar primeiro?

Alguns livros incluem um nível entre os dois primeiros, o **diagnóstico** (“por que aconteceu?”). É o trabalho do analista quando cruza paradas, turno e linha para achar a causa.

> **Teste você 1.1.** Para cada pergunta abaixo, diga **qual defeito de dado a tornaria errada** e em **qual etapa do caminho** (1 a 5) ele nasceria:

1. Quantas leituras de temperatura chegaram ontem?
2. Qual é a temperatura média do sensor SEN00001?
3. Qual colhedora deve parar nos próximos dias?

Dica: pense em duplicatas, nulos, valores sentinela e sensores inexistentes. Não há resposta única; o exercício é treinar o olhar de quem protege o dado.

2. As profissões: onde o engenheiro entra

Slides: Dia 1, 8 a 12 · Estudo: 15 minutos

> **Em uma frase.** O engenheiro garante que o dado exista e seja confiável, todo dia.

Quem faz o quê

	Analista	Engenheiro	Cientista
Pergunta	O que aconteceu e por quê?	O dado chega certo e todo dia?	O que vai acontecer?
Ferramentas	SQL, Excel, Power BI	SQL, Python, pipelines	Python, estatística
Entrega	Relatório e painel	Tabelas confiáveis	Modelos e previsões

O **engenheiro de analytics** fica entre o engenheiro e o analista: modela e testa o dado para a análise, em geral com SQL e dbt. As fronteiras entre os cargos mudam de empresa para empresa.

A rotina do engenheiro

1. Conecta as fontes ao banco.
2. Trata e valida o dado.
3. Mantém o pipeline rodando todo dia.

Entrega: **tabelas confiáveis para quem analisa**. Exemplo na AgroMaq: *leituras de sensor chegando duplicadas*. É exatamente o problema que vamos resolver na prática.

Os três em sequência

Imagine que uma colhedora para no meio do turno:

Quem	O que faz	Resultado
Engenheiro	Garante leituras completas e sem duplicata	Dado confiável
Analista	Cruza paradas, turno e linha para achar o motivo	A causa
Cientista	Usa o histórico para avisar antes da próxima parada	A previsão

> **Atenção.** Sem o primeiro passo bem feito, os outros dois trabalham em cima de dado errado. Você é o primeiro da fila, e o erro que passa por você multiplica.

O que o engenheiro nunca pode esquecer

Princípio	Em uma frase
Confiança	Todo número tem uma origem que você consegue mostrar
Repetibilidade	O pipeline roda de novo e dá o mesmo resultado
Rastreabilidade	O que foi rejeitado fica guardado, com o motivo
Contagem	Entrou, saiu, rejeitou: a conta fecha

Teste você 2.1. Pesquise três vagas reais de engenheiro de dados e anote: quais ferramentas aparecem em todas? Quais pedem nuvem? Quais pedem orquestração (Airflow, por exemplo)? Compare com a tabela acima.

3. Ferramentas: SQL, pandas e a ponte entre eles

Slides: Dia 1, 13 a 17 · Estudo: 120 minutos

> **Em uma frase.** Cada camada do dado tem sua ferramenta: MySQL guarda, SQL consulta, Python trata, Power BI mostra.

Mapa por camada

Camada	Ferramenta	Papel
Guardar	**MySQL**	O banco da fábrica e o data warehouse
Consultar	**SQL**	A língua para perguntar ao banco
Tratar	**Python**	Perfilar, limpar e automatizar
Mostrar	Power BI	Dashboards para decidir
Próximos passos	Airflow, dbt, Spark, nuvem	Quando o volume cresce; só menção

SQL do engenheiro

O molde da consulta é o mesmo de qualquer analista. A diferença é o que você faz com ele: **contar, comparar, validar e transformar**.

```
SELECT   colunas e cálculos
FROM     tabela principal
[JOIN    outra tabela ON condição]
[WHERE   filtro sobre as linhas]
[GROUP BY colunas de agrupamento]
[HAVING  filtro sobre os grupos]
[ORDER BY ordenação]
[LIMIT   quantidade];
```

A ordem **lógica** de execução é `FROM` e `JOIN`, `WHERE`, `GROUP BY`, `HAVING`, `SELECT`, `ORDER BY` e `LIMIT`. Isso explica por que `WHERE` não enxerga apelidos definidos no `SELECT` e por que `HAVING` filtra grupos.

A consulta do slide 14

```
SELECT t.nome AS tipo_sensor,
       COUNT(*) AS sensores
FROM sensor s
JOIN tipo_sensor t
  ON t.id_tipo_sensor = s.tipo_sensor_id
GROUP BY t.nome
```

```
ORDER BY sensores DESC
LIMIT 5;
```

Para cada sensor, traga o nome do seu tipo; agrupe por tipo; conte; mostre os cinco maiores. O resultado esperado é Status Operacional (56), Posicao Linear (44), Temperatura Motor (36), e os dois últimos empatados em 24 (Velocidade de Corte e Vibracao Geral, em ordem que pode variar).

Ferramentas de transformação que você vai usar o tempo todo

Necessidade	Recurso	Exemplo
Condição	`CASE WHEN`	`CASE WHEN valor = -999 THEN NULL ELSE valor END`
Valor ausente	`NULLIF`, `COALESCE`, `IS NULL`	`NULLIF(TRIM(valor), '')`
Texto	`TRIM`, `UPPER`, `LOWER`, `REPLACE`, `CONCAT`	`REPLACE(valor, ',', '.')`
Padrão de texto	`REGEXP`	`data_hora REGEXP '^[0-9]{4}-'`
Data	`STR_TO_DATE`, `DATE_FORMAT`	`STR_TO_DATE(x, '%d/%m/%Y %H:%i')`
Conversão	`CAST`	`CAST(txt AS DECIMAL(10,2))`
Numerar dentro de grupos	`ROW_NUMBER() OVER (...)`	Achar duplicatas

Funções de janela: a ferramenta de deduplicar

`ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)` numera as linhas **dentro de cada grupo**, sem juntá-las. Quem tem número maior que 1 é duplicata:

```
SELECT *
FROM (
  SELECT t.*,
         ROW_NUMBER() OVER (PARTITION BY sensor_id, data_hora
                            ORDER BY linha_arquivo) AS rn
  FROM stg_leitura t
) x
WHERE rn > 1;      -- as sobras: tudo que não é a primeira ocorrência
```

`PARTITION BY` define o grupo (o que é "a mesma leitura"); `ORDER BY` define **qual fica** (a primeira). Você vai usar isso no Capítulo 7.

> **Atenção ao collation.** O MySQL, com a configuração padrão (`utf8mb4_unicode_ci`), **ignora maiúsculas e espaços no fim** ao comparar e agrupar texto. `GROUP BY tipo_maquina` junta `Trator`, `trator`, `TRATOR` e `Trator ` em um grupo só, escondendo o defeito. Para enxergar as variantes, use `GROUP BY BINARY tipo_maquina`.

Python: uma biblioteca para cada etapa

Biblioteca	Para quê	Onde aparece no curso
pandas	Tabelas em memória	Prática P4: perfilar o CSV
NumPy	Cálculo numérico	Base do pandas
SQLAlchemy (com PyMySQL)	Ponte com o MySQL	Ler e gravar no banco pelo Python
Matplotlib	Gráficos rápidos	Conferir o dado
Faker	Dados fictícios	Só os instrutores usaram

Na aula, **você usa o pandas** (P4). SQLAlchemy e Matplotlib podem aparecer como demonstração do instrutor.

pandas 3.0: duas regras que pegam iniciantes

Regra 1: coluna de texto vira `str` (antes era `object`).

```
df["cidade"].dtype      # str  (antes: object)
```

Regra 2: atribuição encadeada deixa de funcionar. Para alterar valores, use `.loc` em um único passo.

```
# ERRADO: não altera df (e o pandas avisa com ChainedAssignmentError)
df[df["valor"] < 0]["valor"] = None

# CERTO
df.loc[df["valor"] < 0, "valor"] = None
```

Receita de perfilamento em pandas

```
import pandas as pd

df = pd.read_csv("leituras_lab.csv", dtype=str)   # tudo texto: nada é "adivinhado"

df.shape                    # (linhas, colunas)
df.info()                   # tipos e não-nulos
df.describe()               # resumo
df.isna().sum()             # nulos por coluna
df.duplicated(subset=["sensor_id", "data_hora", "valor"]).sum()
df["tipo_maquina"].value_counts()
```

A ponte: SQLAlchemy + PyMySQL

```
import pandas as pd
from sqlalchemy import create_engine

engine = create_engine(
    "mysql+pymysql://USUARIO:SENHA@localhost:3306/agromaq_industrial?charset=utf8mb4"
)
```

```
sql = """SELECT t.nome AS tipo_sensor, COUNT(*) AS sensores
         FROM sensor s JOIN tipo_sensor t ON t.id_tipo_sensor = s.tipo_sensor_id
         GROUP BY t.nome ORDER BY sensores DESC LIMIT 5"""
pd.read_sql(sql, engine)                       # banco para o pandas

# pandas para o banco (cria ou substitui uma tabela de staging):
# df.to_sql("stg_leitura_py", engine, if_exists="replace", index=False)
```

Instale com `pip install sqlalchemy pymysql`.

> **Nunca coloque a senha no código que vai para o repositório.** Leia de uma variável de ambiente (`os.environ["DB_SENHA"]`). Hábito de engenheiro desde o primeiro script.

Teste você

Teste você 3.1. Conte as linhas de `maquina` e de `sensor`.

Resultado esperado: 120 e 316.

Teste você 3.2. Rode `SHOW CREATE TABLE leitura_sensor;`. Qual é a chave primária? Por que ela inclui `data_hora`? Quantas partições a tabela tem?

Resultado esperado: chave composta (`id_leitura`, `data_hora`), porque a tabela é **particionada por mês** em `data_hora` e o MySQL exige que a coluna de partição faça parte da chave. São 61 partições (60 meses, de 2025-01 a 2029-12, mais `p_futuro`).

Teste você 3.3. Existe alguma restrição que impeça a mesma leitura (mesmo sensor, mesma hora) de entrar duas vezes em `leitura_sensor`? Qual **índice** existe sobre essas colunas, e ele é único?

Resultado esperado: não há restrição de unicidade; existe o índice `ix_leitura_sensor_data (sensor_id, data_hora)`, que é comum, não único. É por isso que duplicatas podem entrar, e por isso o ETL precisa se proteger.

Teste você 3.4 (janelas). Liste a **primeira máquina de cada linha de produção** (menor `codigo`), usando `ROW_NUMBER()`.

Resultado esperado: 40 linhas (uma por linha de produção). A primeira é `linha_id` 1 com `MAQ0001`; depois 2 com `MAQ0004`, e assim por diante.

Teste você 3.5 (a ponte). No Python, conecte com SQLAlchemy e rode a consulta do slide 14 com `pd.read_sql`. Confira o resultado com o do Workbench.

Resultado esperado: o mesmo. O `DataFrame` tem as colunas `tipo_sensor` (`str`) e `sensores` (`int64`).

4. Ciclo de vida, OLTP × OLAP e ETL × ELT

Slides: Dia 1, 18 a 20 · Estudo: 30 minutos

> **Em uma frase.** O dado percorre um ciclo; um banco opera e outro analisa; e a diferença entre ETL e ELT é **onde a transformação acontece**.

O ciclo de vida do dado

Etapa	O que acontece	Exemplo na AgroMaq
1. Geração	O dado nasce	O sensor mede a temperatura
2. Ingestão	Chega ao banco	A leitura entra no sistema
3. Armazenamento	Fica guardado	Tabela `leitura_sensor`
4. Transformação	Limpeza e regras de negócio	Remover duplicatas, tratar sentinelas
5. Entrega	Alguém consome	Dashboard e modelos

Volte a este ciclo sempre que aprender um conceito novo: ele diz **em que etapa** você está.

OLTP × OLAP

	OLTP (opera)	OLAP (analisa)
Exemplo	`agromaq_industrial`, com 38 tabelas	O data warehouse
Modelo	Normalizado: muitas tabelas, pouca repetição	Estrela: poucas tabelas, com repetição planejada
Escrita	Uma leitura por vez	Cargas em lote
Leitura	Poucas linhas por consulta	Milhões de linhas por consulta
Feito para	Registrar rápido	Responder rápido

Um não substitui o outro: **o ETL leva o dado de um para o outro.** Rodar análises pesadas direto no OLTP competiria com a operação da fábrica.

ETL × ELT

	ETL	ELT
Ordem	Extrair, **Transformar**, Carregar	Extrair, Carregar, **Transformar**
O que faz	Trata **antes** de entrar no destino	Carrega o dado bruto e trata **dentro** do warehouse

	ETL	ELT
Quando brilha	Destino simples; regras de qualidade na entrada	Warehouse poderoso na nuvem; muito volume
No curso	É o que faremos na prática	Só conceito

O ETL não morreu: os dois modelos convivem nas empresas. Na prática, você vai ver pipelines **híbridos**: um pouco de limpeza na entrada e muita transformação dentro do banco.

> **No laboratório você faz um ELT disfarçado de ETL.** O CSV é carregado bruto na staging (o "E" e o "L") e a transformação roda **em SQL dentro do banco** (o "T"). A fronteira é conceitual: o que importa é saber onde cada decisão acontece.

> **Teste você 4.1.** Classifique cada situação como OLTP ou OLAP, e diga em que etapa do ciclo ela fica:

1. O sensor grava uma leitura nova a cada hora.
2. O gerente pede a média de temperatura por unidade nos últimos três meses.
3. Um script remove leituras duplicadas antes de gravar no DW.

5. Dado sujo: reconhecer, medir e listar

Slides: Dia 1, 21 · Práticas P1, P2 e P4 · Estudo: 120 minutos

> **Em uma frase.** Dado real chega sujo. O primeiro trabalho do engenheiro é **medir a sujeira** antes de limpar qualquer coisa.

Seis defeitos clássicos

Defeito	Exemplo	Como aparece na AgroMaq
Nulo	Temperatura vazia	`valor` em branco
Duplicado	A mesma leitura, duas vezes	Mesmo sensor e mesma hora, duas linhas
Sentinela	Um código no lugar do valor	`-999`
Órfão	Leitura de um sensor que não existe	`sensor_id` 9999
Formato	Datas e números escritos de formas diferentes	`15/03/2026 14:00` e `71,5`
Texto solto	Trator, trator e TRATOR	`tipo_maquina` com variantes

> **Alerta: nem todo pico é erro.** Apagar um pico legítimo também é perder dado. O CSV tem um deles; você vai encontrá-lo no Capítulo 7.

A regra da staging: tudo em texto

Na **staging** (a área de entrada), todas as colunas são `VARCHAR`. O motivo: **nada deve ser recusado na entrada**. Se `valor` fosse `DECIMAL`, uma linha com `71,5` ou vazia faria a carga falhar ou, pior, converteria em silêncio. Na staging você só olha o que veio. Nada é corrigido ainda.

Perfilar: o que medir

O que medir	SQL (esqueleto)
Total de linhas	`SELECT COUNT(*) FROM stg_leitura;`
Nulos e vazios	`SUM(valor IS NULL OR TRIM(valor) = '')`
Duplicadas	`ROW_NUMBER() OVER (PARTITION BY ... )` ou `GROUP BY ... HAVING COUNT(*) > 1`
Sentinelas	`WHERE valor IN ('-999')`
Órfãos	`LEFT JOIN agromaq_industrial.sensor ... WHERE ... IS NULL`
Formatos	`WHERE data_hora NOT REGEXP '^[0-9]{4}-[0-9]{2}-[0-9]{2} '` e `valor LIKE '%,%'`

O que medir	SQL (esqueleto)
Variantes de texto	`GROUP BY BINARY tipo_maquina`
Mínimo e máximo	`MIN(...)`, `MAX(...)` em cada coluna

> **Procure pelo que não deveria existir.** Um `MIN(valor)` de `-999` entrega o sentinela; um `MAX(valor)` de 121 pode ser um pico ou um erro. O mínimo e o máximo são as perguntas mais baratas e mais reveladoras.

O mesmo perfilamento em pandas

```
df = pd.read_csv("leituras_lab.csv", dtype=str)

df.isna().sum()                                          # nulos
df.duplicated(subset=["sensor_id", "data_hora", "valor"]).sum()
(df["valor"] == "-999").sum()                            # sentinela
df["valor"].str.contains(",", na=False).sum()            # vírgula decimal
(~df["data_hora"].str.match(r"^\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}$")).sum()
df["tipo_maquina"].value_counts()                        # variantes
```

O objetivo da prática P4 é **bater os números do pandas com os do SQL**. Se não batem, um dos dois está lendo o dado de um jeito diferente.

Teste você

Antes de tudo, prepare a staging: execute `01_criar_staging.sql` e depois carregue o CSV com `02_carregar_csv.sql` (Apêndice A).

> **Teste você 5.1 (P1).** Conecte no MySQL, liste as tabelas e siga a hierarquia: unidade, setor, linha, máquina, sensor.
>
> *Resultado esperado:* 120 máquinas e 316 sensores.
>
> **Teste você 5.2 (P2).** Carregue o CSV na staging (tudo em `VARCHAR`).
>
> *Resultado esperado:* 8.626 linhas em `stg_leitura`.
>
> **Teste você 5.3 (P2).** Escreva consultas para medir cada defeito. Preencha a lista:

Defeito	Resultado esperado
Valores nulos ou vazios	60
Sentinela (`-999`)	45
Órfãos (sensor que não existe no banco)	30
Datas fora do padrão `AAAA-MM-DD HH:MM:SS`	90
Valores com vírgula decimal	50

Defeito	Resultado esperado
Duplicadas pelo **texto** (`sensor_id`, `data_hora`, `valor`)	80
Variantes de `tipo_maquina` (com `BINARY`)	16

Teste você 5.4 (collation). Rode `GROUP BY tipo_maquina` **sem** `BINARY` e depois **com**. Quantos grupos aparecem em cada caso? O que isso ensina?

Resultado esperado: 4 grupos sem `BINARY`; 16 com. O banco, por padrão, esconde a diferença de maiúsculas e de espaço no fim.

Teste você 5.5 (P4). Repita o perfilamento em pandas, com `dtype=str`.

Resultado esperado: os mesmos números da tabela do Teste 5.3. Os órfãos você confere contra a lista de sensores do banco (`isin`).

Teste você 5.6 (a surpresa das datas). Quantas datas **não** são ISO? Se você padronizasse as datas antes de procurar duplicatas, o número de duplicadas **muda**? Para quê?

Resultado esperado: 90 datas fora do padrão. Padronizadas, as duplicadas passam de 80 para **100**: 20 estavam escondidas pelo formato diferente. Você só enxerga isso depois de converter.

Pronto quando: você tem uma **lista de defeitos, um por tipo**, com a contagem de cada. Na staging nada é corrigido ainda.

6. Idempotência: rodar duas vezes não pode duplicar

Slides: Dia 1, 22 · Prática P5 · Estudo: 60 minutos

> **Em uma frase.** Uma carga é idempotente quando repeti-la dá o mesmo resultado. É como o botão do elevador: apertar cinco vezes leva a uma viagem só.

Por que isso importa

Pipelines falham no meio, são reexecutados por engano e rodam em horários que se sobrepõem. Se cada nova execução soma linhas, o primeiro erro de rede vira **dado duplicado** no dashboard de alguém. Um pipeline sem idempotência exige que o engenheiro "adivinhe" o que já entrou.

Teste simples: rodou duas vezes? A **contagem tem que ser a mesma.**

Três ideias que sustentam tudo

Ideia	Em uma frase	Na AgroMaq
Idempotência	Repetir a carga dá o mesmo resultado	Rodar o ETL duas vezes não muda `leitura_tratada`
Staging persistente	O dado bruto fica guardado para recarregar	`stg_leitura` guarda o CSV como veio
Transação	Ou a carga inteira entra, ou nada entra	`START TRANSACTION` … `COMMIT`

A base teórica é a palestra de **Maxime Beauchemin,** ***Functional Data Engineering***: tarefas que podem rodar de novo sem efeito colateral, partições que não mudam e staging persistente. Vale assistir depois.

Os padrões, do mais simples ao mais fino

Padrão	Como funciona	Quando usar	Cuidado
Apagar tudo e recarregar	`DELETE` (ou `TRUNCATE`) e depois `INSERT`	Tabelas pequenas; staging	Janela em que a tabela fica vazia
Apagar o intervalo e reinserir (partição)	`DELETE ... WHERE intervalo` e `INSERT` do mesmo intervalo	**Fatos**, carregados por dia ou lote	O intervalo precisa ser exatamente o mesmo
Atualizar se existe, inserir se é nova (*upsert*)	`INSERT ... ON DUPLICATE KEY UPDATE`	**Dimensões**; cadastros	Exige chave única

Padrão	Como funciona	Quando usar	Cuidado
Restrição de unicidade	`UNIQUE KEY (sensor_id, data_hora)`	Sempre que existir uma chave natural	Vira a rede de proteção, não o plano

`INSERT IGNORE`: cuidado

`INSERT IGNORE` pula linhas que violam a chave única **em silêncio**. Parece idempotente, mas esconde defeitos: uma linha duplicada com valor **diferente** é descartada e ninguém fica sabendo. Prefira `ON DUPLICATE KEY UPDATE` (você escolhe o que atualiza) ou rejeite explicitamente para a quarentena.

```
CREATE TABLE lab_unico (
  sensor_id INT, data_hora DATETIME, valor DECIMAL(10,2),
  UNIQUE KEY uq (sensor_id, data_hora)
);

INSERT INTO lab_unico VALUES (1, '2026-03-01 00:00:00', 70.0);
INSERT INTO lab_unico VALUES (1, '2026-03-01 00:00:00', 71.0);   -- ERRO 1062: Duplicate
entry

INSERT IGNORE INTO lab_unico VALUES (1, '2026-03-01 00:00:00', 71.0);
-- 0 linhas afetadas, valor continua 70: o 71 sumiu sem aviso

INSERT INTO lab_unico VALUES (1, '2026-03-01 00:00:00', 71.0)
ON DUPLICATE KEY UPDATE valor = VALUES(valor);
-- 1 linha na tabela, valor agora 71
```

Transação: tudo ou nada

Uma carga tem vários passos (apagar o lote, inserir, rejeitar). Se ela cair no meio, sem transação você fica com o lote **apagado e só parcialmente reinserido**. Com transação:

```
START TRANSACTION;
DELETE FROM leitura_tratada;
-- ... INSERT ...
COMMIT;      -- confirma tudo
-- ROLLBACK; -- ou desfaz tudo
```

> **Atenção.** Só tabelas **InnoDB** têm transação. Comandos que mudam a estrutura (`TRUNCATE`, `CREATE TABLE`, `DROP TABLE`) **confirmam a transação sozinhos** no MySQL e não podem ser desfeitos com `ROLLBACK`. Dentro de uma transação, use `DELETE`.

Teste você

> **Teste você 6.1 (o erro clássico).** Crie uma tabela `teste_idem` com a mesma estrutura de `stg_leitura` (`CREATE TABLE teste_idem LIKE stg_leitura;`). Carregue o CSV nela **duas vezes seguidas, sem apagar nada**. Quantas linhas ela tem?

Resultado esperado: 17.252, o dobro de 8.626. É a carga não idempotente.

Teste você 6.2 (transação). Na mesma tabela, rode `START TRANSACTION; DELETE FROM teste_idem; SELECT COUNT(*) FROM teste_idem; ROLLBACK; SELECT COUNT(*) FROM teste_idem;`.

Resultado esperado: 0 dentro da transação; 17.252 depois do `ROLLBACK`. Rode de novo com `COMMIT` no lugar de `ROLLBACK` e a contagem fica em 0.

Teste você 6.3 (a staging idempotente). Reescreva a carga da staging para ser idempotente (apagar a staging antes de carregar). Rode duas vezes.

Resultado esperado: 8.626 linhas nas duas execuções.

Teste você 6.4 (a rede de proteção). Reproduza o exemplo `lab_unico`: observe o erro 1062, o `INSERT IGNORE` e o `ON DUPLICATE KEY UPDATE`. Qual valor sobra em cada caso?

Resultado esperado: o `INSERT` simples falha; o `IGNORE` mantém 70; o `UPDATE` deixa 71.

Teste você 6.5 (reflexão). O elevador tem **idempotência** (apertar duas vezes = uma viagem). Dê um exemplo de comando do dia a dia **que não é** idempotente e diga como o tornaria idempotente.

7. ETL: uma regra por defeito

Slides: Dia 1, pipeline e práticas P3, P5 e P6 · Estudo: 120 minutos

> **Em uma frase.** Cada defeito ganha **uma regra explícita**. O que não passa vai para a **quarentena**, com o motivo registrado. No fim, a conta fecha.

O pipeline de hoje em três etapas

Etapa	Cor no slide	O que é
1. Origem	Sapphire	CSV com defeitos: nulos, sentinelas, duplicatas, órfãos, formatos
2. Staging e ETL	(atenção em Gold)	Uma regra por defeito; o que não passa vai para `etl_rejeito`
3. Destino	Emerald	Tabela tratada, validada contra o gabarito, pronta para o Dia 2

Decidir: rejeitar, corrigir ou sinalizar

Para cada defeito, o engenheiro faz uma escolha **de negócio**, não técnica:

Decisão	Quando	Exemplo
Corrigir	A intenção é inequívoca	`71,5` vira `71.5`; `trator` vira `Trator`
Rejeitar (quarentena)	Não dá para saber o valor certo	Valor vazio; sentinela `-999`; sensor inexistente
Manter e sinalizar	Pode ser legítimo	Um pico de temperatura

Regra de ouro: **nunca apague em silêncio.** Se uma linha não entra, ela vai para `etl_rejeito` com o motivo.

As regras do laboratório

Defeito	Regra	Decisão	Motivo em etl_rejeito
Órfão	O `sensor_id` não existe em `agromaq_industrial.sensor`	Rejeitar	`SENSOR_INEXISTENTE`
Nulo	`valor` vazio	Rejeitar	`VALOR_NULO`
Sentinela	`valor = -999`	Rejeitar	`VALOR_SENTINELA`
Formato (data)	Aceitar `AAAA-MM-DD HH:MM:SS` e `DD/MM/AAAA HH:MM`	Corrigir	
Formato (número)	Trocar vírgula por ponto	Corrigir	

Defeito	Regra	Decisão	Motivo em etl_rejeito
Texto solto	`TRIM` e primeira letra maiúscula	Corrigir	
Duplicada	Mesma chave (`sensor_id`, data já padronizada): fica a **primeira**	Rejeitar as demais	`DUPLICADA`

> **A ordem das regras importa.** Primeiro converta e padronize (datas, números), depois valide (nulo, sentinela, órfão), e **só então** procure duplicatas. Se você deduplicar antes de padronizar as datas, deixa passar as duplicatas escritas em outro formato. Uma linha deve receber **um único motivo**: o primeiro que se aplicar.

O desenho do ETL em SQL

Você vai montar a transformação em passos, tudo dentro de **uma transação** e começando por apagar o destino (idempotência, Capítulo 6):

Passo	O que fazer	Ferramenta
1	Apagar `etl_rejeito` e `leitura_tratada`	`DELETE`
2	Criar uma tabela temporária que **padroniza os tipos sem perder a linha de origem** (`linha_arquivo`)	`CREATE TEMPORARY TABLE ... AS SELECT`, `REGEXP`, `STR_TO_DATE`, `REPLACE`, `CAST`
3	Classificar cada linha com **um motivo** (ou nenhum)	`CASE WHEN ... END` e `LEFT JOIN` com `sensor`
4	Marcar as duplicadas **entre as linhas ainda válidas**	`ROW_NUMBER() OVER (PARTITION BY sensor_id, data_hora ORDER BY linha_arquivo)`
5	Inserir os rejeitos em `etl_rejeito` e o resto em `leitura_tratada`	`INSERT ... SELECT`
6	Confirmar e mostrar o **log**: quantas passaram, quantas foram rejeitadas	`COMMIT` e `SELECT`

Dicas para o passo 2:

- Para a data, escolha o formato com `CASE WHEN data_hora REGEXP '^[0-9]{2}/' THEN STR_TO_DATE(..., '%d/%m/%Y %H:%i') ELSE STR_TO_DATE(..., '%Y-%m-%d %H:%i:%s') END`.
- Para o número: `NULLIF(TRIM(valor), '')` primeiro (vazio vira `NULL`), depois `REPLACE(..., ',', '.')`.
- Se o `sensor_id` não for número, o `CAST` pode dar erro; proteja com `REGEXP '^[0-9]+$'`.

> **Escreva e rode um passo de cada vez.** Depois de cada passo, conte as linhas. Se o total não bate com o que você esperava, o defeito está **naquele** passo. Montar tudo de uma vez e depurar o resultado final é o caminho mais longo.

O gabarito do laboratório

Quando o ETL estiver pronto, o log deve mostrar:

	Linhas
Na staging	8.626
Passaram para `leitura_tratada`	**8.391**
Rejeitadas em `etl_rejeito`	**235**

E o resumo dos motivos (`SELECT motivo, COUNT(*) FROM etl_rejeito GROUP BY motivo;`):

Motivo	Linhas
`DUPLICADA`	100
`SENSOR_INEXISTENTE`	30
`VALOR_NULO`	60
`VALOR_SENTINELA`	45

Contas que fecham: 30 + 60 + 45 + 100 = 235 e 8.626 − 235 = 8.391. O resultado tem **seis sensores**, com 1.400, 1.405, 1.400, 1.397, 1.402 e 1.387 linhas (sensores 1, 16, 83, 104, 168 e 244).

O pico que não deveria ser apagado (P6)

Procure o **pico legítimo**. O sensor `16` registra 98,4, 105,2, 112,7, 121,3 e 118,9 °C entre 12h e 16h de 12/03/2026. Uma regra de bolso (“descarte tudo acima de 100 °C”) **apagaria quatro dessas cinco leituras** (105,2, 112,7, 121,3 e 118,9).

> **Regra de negócio vale mais que regra de bolso.** Conheça o dado antes de apagar. O ETL não decide se um pico é falha: ele o **preserva** para quem tem conhecimento da máquina avaliar.

A mesma checagem em pandas (P4)

```
df["valor"] = pd.to_numeric(df["valor"].str.replace(",", "."))
df.loc[df["valor"] == -999, "valor"] = None            # .loc: regra do pandas 3.0

iso = pd.to_datetime(df["data_hora"], format="%Y-%m-%d %H:%M:%S", errors="coerce")
br  = pd.to_datetime(df["data_hora"], format="%d/%m/%Y %H:%M", errors="coerce")
df["data_hora"] = iso.fillna(br)

df["tipo_maquina"] = df["tipo_maquina"].str.strip().str.capitalize()
df["sensor_id"] = df["sensor_id"].astype(int)
```

Depois, `dropna(subset=["valor"])`, filtre os sensores existentes no banco com `isin` e use `drop_duplicates(subset=["sensor_id", "data_hora"], keep="first")`. **Os números do pandas têm que bater com os do SQL.**

Teste você

Teste você 7.1 (P3). Escreva o ETL em SQL seguindo os seis passos.

Resultado esperado: o log com 8.626, 8.391 e 235, e a tabela de motivos acima.

Teste você 7.2 (idempotência). Rode o ETL **duas vezes** seguidas.

Resultado esperado: o log idêntico nas duas, sem somar linhas.

Teste você 7.3 (P5, gabarito). Valide `leitura_tratada` com as somas de verificação: `COUNT(*)`, `SUM(valor)`, `MIN(valor)`, `MAX(valor)` e as datas mínima e máxima.

Resultado esperado: 8.391 linhas; soma de `valor` **643.856,20**; mínimo 62,60; máximo **121,30**; de 2026-02-01 00:00:00 a 2026-03-31 23:00:00; `tipo_maquina` com 4 valores distintos (`Colhedora`, `Plantadora`, `Pulverizador`, `Trator`); status 8.351 `VALIDA` e 40 `SUSPEITA`.

Teste você 7.4 (P6, o pico). Consulte as leituras do sensor `16` no dia 12/03/2026, entre 12h e 16h.

Resultado esperado: as 5 leituras estão em `leitura_tratada`: 98,40 / 105,20 / 112,70 / 121,30 / 118,90.

Teste você 7.5 (P4, pandas). Faça o tratamento em pandas e compare.

Resultado esperado: 8.391 linhas tratadas e as mesmas contagens por sensor.

Teste você 7.6 (a quarentena). Escolha 3 linhas de `etl_rejeito` e volte ao CSV para encontrar cada uma pelo número da linha de origem. A explicação do motivo faz sentido?

Resultado esperado: cada rejeito aponta para uma linha do CSV (`linha_arquivo`) cujo defeito corresponde ao motivo registrado.

Pronto quando: o log mostra quantas passaram e quantas foram para `etl_rejeito`, cada rejeito tem motivo, e a **segunda carga não muda a contagem**.

8. Modelo dimensional: desenhar antes de carregar

Slides: Dia 1, 23 · Dia 2, 3 e 4 · Estudo: 60 minutos

> **Em uma frase.** Fato é o que se mede; dimensão é o contexto para fatiar. A primeira decisão é a **granularidade**: uma linha da fato representa o quê?

O DW da AgroMaq

O data warehouse do curso é uma **estrela** com duas fatos e cinco dimensões:

Tabela	Tipo	Granularidade	Observação
`fato_leitura_sensor`	Fato	Uma linha por sensor por hora	O volume completo tem cerca de 2,15 milhões de linhas
`fato_producao`	Fato	Uma linha por ordem ou lote	Granularidade exata confirmada no DDL do DW
`dim_tempo`	Dimensão	Uma linha por período de tempo	
`dim_sensor`	Dimensão	Uma linha por sensor	Larga: traz máquina, linha e unidade
`dim_maquina`	Dimensão	Uma linha por máquina	
`dim_linha`	Dimensão	Uma linha por linha de produção	
`dim_produto`	Dimensão	Uma linha por produto	
`dim_cliente`	Dimensão	Uma linha por cliente	Só no **desafio de vendas**

Decisões de modelagem

1. Granularidade primeiro. Escreva em uma frase: *"cada linha da fato é … "*. Se a frase tiver um "ou", tem duas granularidades misturadas.

2. A dimensão achata a hierarquia. No OLTP: sensor, máquina, linha, unidade, quatro tabelas. No DW, `dim_sensor` é **uma linha larga** com tudo isso. Você troca espaço (repetição) por velocidade e simplicidade na leitura. A repetição é planejada: nada se grava duas vezes à mão, o ETL preenche.

3. Chave natural e chave substituta.

	Chave natural	Chave substituta
O que é	O código do sistema de origem	Um número criado pelo próprio DW

	Chave natural	Chave substituta
Exemplo	`id_sensor_origem`	`sk_sensor`
Por que as duas	A natural permite reconhecer o mesmo sensor em cargas futuras (é a chave do *upsert*)	A substituta liga fato e dimensão sem depender de regras do sistema de origem

4. A dimensão de tempo e a tabela de datas do Power BI. Uma dimensão de tempo **por hora** repetiria cada data 24 vezes. O Power BI exige uma data **única por linha** para usar funções de tempo (mês anterior, acumulado no ano). Por isso, no laboratório, `dim_tempo` tem **uma linha por dia**, e `hora` e `turno` ficam na fato. É uma decisão que o consumidor do dado agradece.

O DW do laboratório

O laboratório tem uma versão pequena do DW. O script `03_dw_ddl.sql` cria as cinco tabelas; os trechos principais:

```
CREATE TABLE dim_sensor (
    sk_sensor         INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,   -- chave substituta
    id_sensor_origem  INT UNSIGNED NOT NULL,                     -- chave natural
    codigo_sensor     VARCHAR(30)  NOT NULL,
    tipo_sensor       VARCHAR(100) NOT NULL,
    codigo_maquina    VARCHAR(30)  NOT NULL,
    nome_maquina      VARCHAR(100) NOT NULL,
    nome_linha        VARCHAR(100) NOT NULL,
    categoria         VARCHAR(50)  NOT NULL,
    unidade           VARCHAR(100) NOT NULL,
    faixa_min         DECIMAL(10,2) NULL,    -- definição do laboratório
    faixa_max         DECIMAL(10,2) NULL,
    UNIQUE KEY uq_dim_sensor_origem (id_sensor_origem)           -- base do upsert
);

CREATE TABLE dim_tempo (
    sk_tempo     INT UNSIGNED PRIMARY KEY,         -- AAAAMMDD
    data         DATE NOT NULL,
    ano SMALLINT NOT NULL, mes TINYINT NOT NULL, nome_mes VARCHAR(12) NOT NULL,
    dia TINYINT NOT NULL, dia_semana VARCHAR(15) NOT NULL,
    UNIQUE KEY uq_dim_tempo_data (data)
);

CREATE TABLE fato_leitura_sensor (
    sk_tempo        INT UNSIGNED NOT NULL,         -- dia
    hora            TINYINT NOT NULL,              -- 0 a 23
    turno           VARCHAR(10) NOT NULL,          -- Manha, Tarde, Noite
    sk_sensor       INT UNSIGNED NOT NULL,
    valor           DECIMAL(10,2) NOT NULL,
    status_leitura  VARCHAR(10) NOT NULL,
    fora_faixa      TINYINT NOT NULL,              -- 1 = fora da faixa, 0 = dentro
    PRIMARY KEY (sk_tempo, hora, sk_sensor),       -- a granularidade, em forma de chave
    FOREIGN KEY (sk_tempo)  REFERENCES dim_tempo  (sk_tempo),
```

```
    FOREIGN KEY (sk_sensor) REFERENCES dim_sensor (sk_sensor)
);
```

> **A chave primária da fato é a granularidade escrita em SQL.** (`sk_tempo`, `hora`, `sk_sensor`) significa "uma linha por sensor por hora". Se alguém tentar inserir duas, o banco recusa. É uma rede de proteção que custa uma linha.

`dim_linha` e `dim_maquina` completam o conjunto; elas existem para a futura `fato_producao`, e `dim_maquina` aponta para `dim_linha` pela chave substituta `sk_linha`.

De onde vem cada coluna

Coluna do DW	Origem no banco operacional
`dim_sensor.codigo_sensor`, `id_sensor_origem`	`sensor.codigo`, `sensor.id_sensor`
`dim_sensor.tipo_sensor`	`tipo_sensor.nome`, via `sensor.tipo_sensor_id`
`dim_sensor.nome_maquina`, `codigo_maquina`	`maquina.nome`, `maquina.codigo`, via `sensor.maquina_id`
`dim_sensor.nome_linha`	`linha_producao.nome`, via `maquina.linha_id`
`dim_sensor.categoria`	`categoria_produto.nome`, via `linha_producao.categoria_id`
`dim_sensor.unidade`	`unidade.nome`, via `categoria_produto.unidade_id`
`fato.valor`, `status_leitura`	`leitura_tratada` (já limpo)
`fato.fora_faixa`	**Regra de negócio** calculada na carga: `valor` fora de `faixa_min` e `faixa_max`
`fato.turno`	Calculado da hora: Manhã 6 a 13h, Tarde 14 a 21h, Noite 22 a 5h

Teste você

> **Teste você 8.1.** Escreva, em **uma frase cada**, a granularidade de `fato_leitura_sensor` e de `fato_producao`. Depois explique por que a frase da segunda não deve ter a palavra "ou".
>
> **Teste você 8.2.** Execute `03_dw_ddl.sql`. Rode `SHOW TABLES;` e `SHOW CREATE TABLE fato_leitura_sensor;`.
>
> *Resultado esperado:* além de `stg_leitura`, `etl_rejeito` e `leitura_tratada`, aparecem 5 tabelas novas: `dim_linha`, `dim_maquina`, `dim_sensor`, `dim_tempo` e `fato_leitura_sensor`. A fato tem duas chaves estrangeiras e a chave primária de três colunas.
>
> **Teste você 8.3.** Quais tabelas do OLTP você precisa juntar (`JOIN`) para preencher uma linha de `dim_sensor`? Quantas linhas essa junção deve devolver?

Resultado esperado: `sensor`, `tipo_sensor`, `maquina`, `linha_producao`, `categoria_produto` e `unidade` (seis tabelas); **316** linhas, uma por sensor.

Teste você 8.4. Por que a `dim_sensor` precisa de uma chave substituta se já existe `id_sensor`? Dê dois motivos.

Teste você 8.5 (desenho). Rascunhe, em papel, a estrela do laboratório: a fato no centro e as dimensões ao redor, escrevendo nas ligações **qual coluna liga a qual**. Compare com o slide 8 do Dia 2.

9. Carga no DW: dimensões, fatos e reconciliação

Slides: Dia 2, 5 · Práticas E1 a E3 · Estudo: 120 minutos

> **Em uma frase.** Dimensões se atualizam (*upsert*); fatos se recarregam por partição; e no fim a conta fecha: **linhas da staging menos rejeitos igual às linhas na fato**.

A ordem de carga

Dimensões primeiro, fato depois. A fato precisa das chaves substitutas, que só existem depois que as dimensões foram carregadas.

Ordem	Tabela	Padrão
1	`dim_linha`	*Upsert* pela chave natural `id_linha_origem`
2	`dim_maquina`	*Upsert*; busca `sk_linha` em `dim_linha`
3	`dim_sensor`	*Upsert*; junta seis tabelas do OLTP
4	`dim_tempo`	*Upsert*; gera os dias do período
5	`fato_leitura_sensor`	Apaga o intervalo e reinsere

Tudo dentro de **uma transação**.

Dimensões: atualiza se existe, insere se é nova

```
INSERT INTO dim_maquina (id_maquina_origem, codigo, ...)
SELECT m.id_maquina, m.codigo, ...
FROM agromaq_industrial.maquina m
JOIN dim_linha dl ON dl.id_linha_origem = m.linha_id
ON DUPLICATE KEY UPDATE codigo = VALUES(codigo), ...;
```

Este é um **esquema**, não o script final: os `...` são colunas que você completa. O `ON DUPLICATE KEY UPDATE` age quando a chave única (`id_maquina_origem`) já existe: em vez de falhar ou duplicar, ele **atualiza** a linha. Se o nome de uma máquina mudou no OLTP, a dimensão acompanha; se nada mudou, nada muda. Rodar duas vezes dá o mesmo resultado.

> **A chave substituta fica estável.** Em um *upsert*, as linhas que já existem mantêm o `sk_*` original. O contador de `AUTO_INCREMENT` pode avançar e deixar **buracos** na sequência (a próxima `sk_sensor` nova pode pular números). Isso é normal e não afeta nada: o que importa é que a ligação entre fato e dimensão continue a mesma.

Dica para `dim_tempo`: gerar os dias

Você precisa de uma linha por dia entre a primeira e a última leitura tratada. No MySQL 8, uma CTE recursiva resolve:

```
WITH RECURSIVE dias (dt) AS (
    SELECT DATE(MIN(data_hora)) FROM leitura_tratada
    UNION ALL
    SELECT dt + INTERVAL 1 DAY FROM dias
    WHERE dt < (SELECT DATE(MAX(data_hora)) FROM leitura_tratada)
)
SELECT dt FROM dias;
```

Para os nomes em português use `ELT(MONTH(dt), 'Janeiro', 'Fevereiro', ...)` e `ELT(WEEKDAY(dt) + 1, 'Segunda', 'Terca', ...)`. A chave: `CAST(DATE_FORMAT(dt, '%Y%m%d') AS UNSIGNED)`. A CTE recursiva do MySQL 8 aceita 1.000 níveis por padrão; para períodos maiores, ajuste `cte_max_recursion_depth`.

Dica para `dim_sensor`: a faixa

As colunas `faixa_min` e `faixa_max` **não existem no banco operacional**. Para o laboratório, defina: sensores do tipo `Temperatura Motor` têm faixa de **50 a 95 °C**; os demais ficam com `NULL`. Em um projeto real, essa regra viria de quem conhece a operação.

Fato: apaga o intervalo e reinsere

A fato é carregada por **partição**: o intervalo do lote é apagado e reinserido. Se o ETL rodar de novo para o mesmo intervalo, o resultado é idêntico.

```
-- 1. apaga o intervalo
DELETE FROM fato_leitura_sensor WHERE sk_tempo BETWEEN @inicio AND @fim;
-- 2. reinsere o mesmo intervalo
INSERT INTO fato_leitura_sensor (...) SELECT ... FROM leitura_tratada ...;
```

Pontos de atenção na fato:

Ponto	Como fazer
Achar `sk_sensor`	`JOIN dim_sensor ON id_sensor_origem = leitura_tratada.sensor_id`
Achar `sk_tempo`	`JOIN dim_tempo ON data = DATE(leitura_tratada.data_hora)`
`hora`	`HOUR(data_hora)`
`turno`	`CASE`: Manhã 6 a 13h, Tarde 14 a 21h, Noite as demais
`fora_faixa`	`1` se o valor está fora de `faixa_min` e `faixa_max`; `0` se não; `0` se a faixa é nula
Intervalo	`@inicio` e `@fim` = menor e maior `sk_tempo` carregados

Reconciliação: a conta que fecha

```
SELECT (SELECT COUNT(*) FROM stg_leitura)            AS staging,
       (SELECT COUNT(*) FROM etl_rejeito)            AS rejeitos,
       (SELECT COUNT(*) FROM fato_leitura_sensor)    AS na_fato;
-- staging - rejeitos = na_fato
```

Mais duas verificações de integridade:

- **Fatos sem dimensão:** `LEFT JOIN` da fato com cada dimensão, procurando `IS NULL`. Deve dar zero.
- **Chaves nulas:** não deve haver nenhuma (as chaves estrangeiras já impedem).

Reconciliar **toda vez** é o hábito que separa um pipeline confiável de um que "parece certo".

Teste você

Pré-requisito: o ETL do Capítulo 7 já rodou (`leitura_tratada` com 8.391 linhas) e o DDL do Capítulo 8 já foi executado.

Teste você 9.1 (E1, dimensões). Escreva e rode o script das dimensões, na ordem certa, com *upsert*.

Resultado esperado: `dim_linha` 40, `dim_maquina` 120, `dim_sensor` 316, `dim_tempo` 59 linhas.

Teste você 9.2 (E2, fato). Escreva e rode a carga da fato, com as chaves substitutas.

Resultado esperado: `fato_leitura_sensor` com **8.391** linhas, nenhuma chave nula e `SUM(fora_faixa)` igual a **74**.

Teste você 9.3 (E3, idempotência e reconciliação). Rode **tudo de novo** (dimensões e fato) e reconcilie.

Resultado esperado: as mesmas contagens de antes; e a conta: 8.626 (staging) menos 235 (rejeitos) é igual a 8.391 (fato).

Teste você 9.4 (chaves estáveis). Depois da segunda carga, rode `SELECT MIN(sk_sensor), MAX(sk_sensor) FROM dim_sensor;`. Algum `sk_sensor` mudou?

Resultado esperado: de 1 a 316, sem mudança: o *upsert* preserva as chaves.

Teste você 9.5 (a transação). Provoque um erro de propósito no meio da carga da fato (por exemplo, escrevendo mal o nome de uma coluna no `INSERT`) dentro de uma transação, e depois dê `ROLLBACK`. Qual é o estado da fato?

Resultado esperado: a fato **volta ao que era antes da transação**, com as 8.391 linhas anteriores, porque o `DELETE` do intervalo também foi desfeito.

Teste você 9.6 (valide o negócio). Quantas leituras fora da faixa existem por categoria e por turno? Compare com os valores esperados.

Resultado esperado: 74 no total: Colhedora 69 (todas no turno Noite, do sensor `SEN00104`) e Trator 5 (o pico de 12/03 do sensor `SEN00016`, 2 na Manhã e 3 na Tarde).

Pronto quando: as contagens batem com o gabarito, **a segunda carga não muda nada** e a reconciliação fecha. Travou? Na aula, haverá um DW pré-carregado de reserva.

10. Power BI, visto do DW

Slides: Dia 2, 6 a 9 e B1 · Estudo: 30 minutos

> **Em uma frase.** O Power BI é o cliente do seu DW. Quanto melhor o DW, menos trabalho (e menos erro) no Power BI.

Você não vai construir dashboards neste curso, mas **vai entregar o DW que alimenta o dashboard**. Entender o que o Power BI espera do seu modelo é parte do trabalho.

As quatro etapas

Etapa	O que acontece	Quem pesa mais
Conectar	Obter dados do banco ou de arquivos	Engenharia (acesso, driver, desempenho)
Transformar	Power Query: tipos, filtros, mesclas	Os dois
Modelar	Relacionamentos e medidas	Engenharia na estrela; análise nas medidas
Visualizar	Gráficos e filtros	Análise

Na aula ficamos no **Power BI Desktop**, onde se constrói o modelo. O **Power BI Service** é onde se publica e compartilha com a empresa; ele é apenas citado.

Power Query × DW: onde cada coisa mora

O Power Query é o "ETL leve" do analista: uma lista de **passos aplicados**, na ordem. Ele é ótimo para ajustes, mas **não é lugar de regra de negócio**.

Fica no Power Query	Fica no DW
Ajuste de apresentação (renomear coluna para o relatório)	Limpeza pesada (nulos, sentinelas, duplicadas)
Escolher quais colunas carregar	Regra de negócio (`fora_faixa`, `turno`)
Tipo de coluna, quando o conector erra	Chaves substitutas e relacionamentos
Mesclas pontuais	Dimensões prontas e conformes

A pergunta-guia do slide: **"isso cabe aqui ou deveria estar no DW?"** Se a regra está no Power Query, ela vale para um relatório só. Se está no DW, vale para todos, e é testada uma vez.

> **O risco de duas verdades.** Se `turno` é calculado no DW e de novo no Power Query de outro relatório, cedo ou tarde os dois divergem. Regra de negócio em **um** lugar.

O DW já vem no formato que o Power BI quer

O Power BI trabalha melhor com **modelo estrela**: fatos no centro, dimensões ao redor, relacionamentos **um para muitos** (uma máquina, muitas leituras) e filtro que vai **da dimensão para a fato**.

Quando você desenha o DW como no Capítulo 8, o modelo do Power BI praticamente se monta sozinho: as chaves `sk_*` viram os relacionamentos. Foi exatamente por isso que a granularidade e as chaves foram decididas antes.

Duas coisas do lado da engenharia que fazem diferença:

- `dim_tempo` **em granularidade diária**, com a coluna `data` do tipo `DATE`. O Power BI pede uma tabela de datas contínua, sem buracos, para marcar como **tabela de datas**. Por isso `hora` e `turno` ficam na fato.
- **Nomes legíveis**. `nome_maquina` é melhor que `nm_maq`. Quem faz o relatório lê o que você nomeou.

Medida × coluna calculada

	Coluna calculada	Medida
Quando calcula	Na carga, linha a linha	Na hora, conforme o filtro
Onde fica	Na memória, ocupando espaço	Só a fórmula
Quando usar	Atributo fixo da linha	Totais, taxas, comparações

No volume completo (2,15 milhões de leituras) uma coluna calculada custa caro. Por isso o padrão é **medida**. E, do seu lado, se um atributo fixo pode ser calculado **no DW** (como `fora_faixa`), calcule lá, uma vez, e deixe o Power BI só somar.

Importar × DirectQuery

	Importação	DirectQuery
Onde fica o dado	Dentro do arquivo do Power BI	No banco; cada visual consulta
Velocidade	Rápida	Depende do banco
Atualização	Agendada	Sempre atual

Na aula usamos **Importação** sobre o recorte dos alunos. Em um projeto real, a escolha depende do volume e da frescura exigida.

B1: conectar o Power BI ao DW

1. **Obter dados › Banco de dados MySQL**.
2. Servidor `localhost` e banco do DW.

3. Marque as dimensões e as fatos e carregue.

Pronto quando: as tabelas aparecem no painel de dados. Se o conector falhar (falta o Oracle MySQL Connector/NET, ou política da máquina), o **plano B** é exportar as tabelas do DW em CSV e importar por **Texto/CSV**. No laboratório, os CSVs já estão em `lab_dw_csv/`. Os CSVs usam **ponto decimal**; se o Power BI em português ler `71.5` como `715`, converta a coluna com **Alterar tipo › Usando a localidade** e escolha **Inglês (Estados Unidos)**.

Teste você

Teste você 10.1 (B1). Conecte o Power BI Desktop ao banco `agromaq_lab` e carregue as 5 tabelas do DW.

Resultado esperado: as 5 tabelas no painel de dados (`dim_linha`, `dim_maquina`, `dim_sensor`, `dim_tempo`, `fato_leitura_sensor`). Se o conector falhar, repita o exercício importando os CSVs da pasta `lab_dw_csv`.

Teste você 10.2 (relacionamentos). Na visão de **Modelo**, confira os relacionamentos criados. Qual é o lado "um" e o lado "muitos" de cada um? Em que direção vai o filtro?

Resultado esperado: cada dimensão no lado "um", a fato no lado "muitos"; o filtro vai da dimensão para a fato. Se algum relacionamento não foi criado, crie-o arrastando `sk_*` da dimensão para a fato.

Teste você 10.3 (a tabela de datas). Marque `dim_tempo` como tabela de datas, usando a coluna `data`. Funcionou? Agora imagine que `dim_tempo` tivesse uma linha por hora: o que aconteceria?

Resultado esperado: marcar funciona porque há **uma linha por dia, sem repetição**. Com uma linha por hora, a coluna de data teria valores repetidos e o Power BI recusaria; esse é o motivo de `hora` ficar na fato.

Teste você 10.4 (confira com o DW). Crie uma medida `Leituras = COUNTROWS(fato_leitura_sensor)` e uma `Fora da faixa = SUM(fato_leitura_sensor[fora_faixa])`. Compare com o SQL.

Resultado esperado: **8.391** e **74**, os mesmos números da reconciliação do Capítulo 9. Se um valor diferir, o problema está na carga, não no Power BI.

Pronto quando: o Power BI mostra 8.391 leituras e 74 fora da faixa, iguais ao DW. Esse é o teste que o engenheiro faz antes de entregar o DW.

11. IA na engenharia

Slides: Dia 1, 24 a 26 · Dia 2, 16 · Estudo: 40 minutos

> **Em uma frase.** Delegue a escrita, não a conferência. A IA escreve rápido; quem responde pelo resultado é você.

O padrão: peça, revise, teste, compare

A IA é boa para o primeiro rascunho de um SQL, de uma regex, de um script. Ela não sabe o que você tem no banco, a menos que você conte, e não responde pelo número que entrega.

Uma boa pergunta tem quatro partes: **contexto** (o schema), **tarefa**, **restrição** e **formato**.

> Tenho a tabela `stg_leitura(linha_arquivo, sensor_id, data_hora, valor, status_leitura, tipo_maquina)`, tudo em texto. Escreva um SQL para MySQL 8 que ache leituras duplicadas. Explique cada linha em uma frase.

O checklist

Passo	O que fazer
1. Schema como contexto	Passe nomes e tipos reais das colunas
2. Revisar linha a linha	A tabela existe? O agrupamento faz sentido?
3. Testar numa amostra	Rode num recorte pequeno antes do conjunto inteiro
4. Comparar contagens	Antes e depois, e contra um número que você já conhece
5. Nada sensível	Não cole dado real de cliente no chat

No curso o dado é fictício, mas **a regra se ensina**.

Demo: a IA acha duplicadas, mas quantas?

A resposta típica de uma IA para o pedido acima é:

```
SELECT sensor_id, data_hora, COUNT(*) AS vezes
FROM stg_leitura
GROUP BY sensor_id, data_hora
HAVING COUNT(*) > 1;
```

O SQL está correto e roda. Mas **compare com o que você sabe**. No laboratório, a consulta devolve **80 grupos**. E o gabarito do ETL (Capítulo 7) rejeita **100** linhas como `DUPLICADA`. Onde estão as 20 de diferença?

Estão nas cópias cuja data veio no formato `dd/mm/aaaa hh:mm`. Como a staging é texto, `'01/02/2026 03:00'` e `'2026-02-01 03:00:00'` são textos diferentes e **não se agrupam**. A consulta da IA estava

certa para o que você pediu, e errada para o que você queria. É o risco da "métrica errada com cara de certa".

> **Quem descobriu?** Não foi a IA, foi a comparação com um número conhecido (100 do gabarito). É por isso que o passo 4 do checklist existe.

Dado ruim derruba a IA também

O mesmo defeito que atrapalha o relatório vira resposta errada em um modelo ou assistente.

Dado sujo	Assistente ou modelo	Resposta
Um `-999` no lugar do valor	Calcula média ou treina em cima	Uma média torta, dita com confiança
Duplicadas	Contam em dobro	Totais inflados
Órfãos	Perdem o contexto	Linhas sem máquina ou linha

Qualidade de dado é pré-requisito de IA. O pipeline de hoje protege a IA de amanhã.

Os quatro padrões de IA em BI

Nos dashboards, a IA aparece como **pergunta vira consulta**, **narrativa automática**, **detecção de anomalia** e **assistente de fórmula**. O Power BI Desktop traz recursos como Key Influencers, Decomposition Tree, Smart Narrative, Q&A e Quick Insights; o Copilot exige capacidade paga e ativação pelo administrador. Para o engenheiro, o ponto é outro: **todo recurso desses é tão bom quanto o DW por baixo**. Um `turno` mal calculado vira uma "descoberta" falsa do Key Influencers.

Teste você

Teste você 11.1 (a demo). Peça a uma IA o SQL de duplicadas, dando o schema da `stg_leitura`. Rode no laboratório.

Resultado esperado: **80 grupos**, cada um com `vezes = 2`. Se a IA devolver outra coisa, revise: nome de tabela inventado? Colunas que não existem? Agrupamento por coluna errada?

Teste você 11.2 (compare com o gabarito). O ETL do Capítulo 7 rejeitou 100 linhas como `DUPLICADA`. Explique por que a consulta da IA enxerga só 80. Depois adapte-a para enxergar as 100, padronizando a data antes de agrupar.

Resultado esperado: as 20 cópias estão com a data em formato brasileiro, que é outro texto. Com a data convertida (`STR_TO_DATE` para os dois formatos) e o agrupamento feito sobre a data padronizada, os excedentes somam **100** (`SUM(n - 1) = 100`).

Teste você 11.3 (delegue e confira). Peça à IA para escrever o ETL inteiro do Capítulo 7. Rode, e **compare com o gabarito**: 8.391 linhas tratadas e os rejeitos por motivo (30, 60, 45, 100).

Resultado esperado: é comum a IA acertar a maior parte e errar um detalhe (por exemplo, remover duplicadas **antes** de padronizar a data, ou apagar o pico de 12/03). Registre onde divergiu e por quê. Esse é o exercício mais valioso do capítulo.

Teste você 11.4 (dado ruim, IA ruim). Calcule a média de `valor` na staging (troque a vírgula por ponto e use `CAST`), primeiro deixando o `-999` entrar e depois só com valores válidos (sem vazio e sem `-999`). Qual é a diferença?

Resultado esperado: cerca de **71,07** com os 45 sentinelas contra cerca de **76,72** sem eles. Uma diferença de mais de 5 °C só por causa de 45 linhas. É a prova concreta de que dado sujo vira resposta errada dita com confiança.

Pronto quando: você consegue explicar, em duas frases, por que a IA pode escrever o ETL mas não pode assinar o resultado.

12. Carreira e próximos passos

Slides: Dia 2, 20, 21 e 23 · Estudo: 20 minutos

O caminho que você percorreu

Dado bruto, pipeline, data warehouse, dashboard, insight. Cada etapa foi de alguém, e a decisão final só é boa se todas estiverem boas. O engenheiro cuida das três primeiras.

Trilhas e certificações iniciais

Trilha	Certificação	O que cobre
Engenharia no Fabric	**DP-700**	Pipelines e engenharia de dados no Microsoft Fabric (atualização da versão em inglês em 19/out/2026)
Análise e engenharia no Fabric	**DP-600**	Modelagem e análise no Microsoft Fabric
Engenharia em outras nuvens	**AWS Data Engineer Associate (DEA-C01)**, **Databricks Data Engineer Associate**, **Google Professional Data Engineer**	Engenharia de dados em AWS, Databricks e Google Cloud
Análise	**PL-300**	Power BI, SQL e DAX (útil para entender o seu cliente)

> **Confira antes de se inscrever.** Nomes, preços e exigências de certificação mudam. Veja a página oficial de cada uma.

O que estudar depois, em ordem

1. **SQL de verdade.** Refaça os capítulos 5, 7 e 9 sem consultar; depois aprenda funções de janela e índices.
2. **Python para dados.** `pandas`, `sqlalchemy`, e organizar o ETL em scripts com *logs*.
3. **Orquestração.** Um agendador como o **Airflow** roda o pipeline todo dia e avisa quando falha.
4. **Transformação em SQL versionada.** Ferramentas como o **dbt** organizam o "T" do ELT com testes.
5. **Processamento em escala. Spark** para quando o dado não cabe mais em uma máquina.
6. **Uma nuvem.** Escolha uma (Azure, AWS ou Google) e faça um projeto de ponta a ponta.

Não precisa aprender tudo. Cada item resolve um problema que você já viu em escala menor: **repetir** (orquestração), **testar** (dbt), **crescer** (Spark e nuvem).

Perguntas para pensar

- O que mais quebra no mundo real?
- O que você deixaria a IA fazer sozinha, e o que você sempre conferiria?
- Se o pipeline falhasse às 3 da manhã, quem seria avisado e como?

> **A mensagem final.** Ferramentas mudam a cada ano. Idempotência, reconciliação, granularidade e a desconfiança saudável do dado não mudam. Quem entende o dado troca de ferramenta sem sofrer.

Apêndice A. Montando o laboratório

Tempo estimado: 40 a 60 minutos na primeira vez.

Arquivos do laboratório

Arquivo	Para quê
`leituras_lab.csv`	O CSV com defeitos plantados (8.626 linhas)
`01_criar_staging.sql`	Cria o banco `agromaq_lab` e as tabelas de staging, rejeitos e tratada
`02_carregar_csv.sql`	Carrega o CSV na staging (`LOAD DATA LOCAL INFILE`)
`03_dw_ddl.sql`	Cria as tabelas do DW do laboratório
`lab_dw_pronto.sql`	O DW do laboratório já carregado (reserva e conferência)
`lab_dw_csv/`	As mesmas tabelas do DW em CSV (plano B do Power BI e do pandas)
`02_schema.sql` e `03_populacao_base.sql`	O banco `agromaq_industrial` (fornecidos pelos instrutores)

Passo a passo

1. Instale o MySQL 8 e o MySQL Workbench. Anote a senha do usuário `root`.

2. Crie o banco da AgroMaq.

```
mysql -u root -p < 02_schema.sql
mysql -u root -p agromaq_industrial < 03_populacao_base.sql
```

Confira: `SELECT COUNT(*) FROM maquina;` deve dar 120 e `SELECT COUNT(*) FROM sensor;` deve dar 316.

3. Crie a staging e carregue o CSV. O `LOAD DATA LOCAL INFILE` precisa estar habilitado nos dois lados:

```
mysql -u root -p --local-infile=1
```

e, dentro do cliente, `SET GLOBAL local_infile = 1;`. Depois execute `01_criar_staging.sql` e `02_carregar_csv.sql` (no segundo, ajuste o caminho do CSV se preciso).

Confira:

```
USE agromaq_lab;
SELECT COUNT(*) FROM stg_leitura;   -- 8626
```

4. Escreva o ETL (Capítulo 7), o DDL (Capítulo 8) e a carga (Capítulo 9). Esse é o exercício. Se travar, o arquivo `lab_dw_pronto.sql` entrega o DW já carregado para você conferir seus números.

5. Prepare o Python. Python 3.11 ou mais novo; em um terminal, dentro da pasta do laboratório:

```
python -m venv .venv
.venv\Scripts\activate          (Windows)
pip install pandas sqlalchemy pymysql
python -c "import pandas; print(pandas.__version__)"
```

A versão deve começar com `3`.

Se algo der errado

Sintoma	Causa provável	O que fazer
`Access denied`	Senha ou usuário errados	Use o usuário `root` e a senha da instalação
`Loading local data is disabled`	`local_infile` desligado	`SET GLOBAL local_infile = 1;` e conecte com `--local-infile=1`
`Unknown database 'agromaq_lab'`	O passo 3 não rodou	Execute `01_criar_staging.sql`
Contagem de linhas diferente de 8626	CSV aberto e salvo em outro programa	Use o arquivo original; quebras de linha e aspas importam
`Cannot add or update a child row`	Ordem de carga errada	Dimensões antes da fato; apague a fato antes de recarregar dimensões
`FutureWarning` ou `ChainedAssignmentError` no pandas	Código de versão antiga	Use `.loc` (Capítulo 3)
Power BI não lista "MySQL"	Falta o Connector/NET	Instale o conector e **reinicie** o Power BI; plano B: CSVs

Apêndice B. Glossário

Termo	Significado
Chave natural	O código que vem do sistema de origem (`id_*_origem`)
Chave substituta	Número criado pelo DW para ligar fato e dimensão (`sk_*`)
CTE	*Common Table Expression*: consulta nomeada com `WITH`
Dado sentinela	Código (como `-999`) no lugar de um valor
Data warehouse (DW)	Banco organizado para análise, com fatos e dimensões
Dimensão	Tabela que descreve o contexto: quando, onde, quem
ELT	Carrega o dado bruto e trata dentro do destino
ETL	Extrai, transforma e carrega, tratando antes de entrar no destino
Fato	Tabela do que aconteceu, com números
Granularidade	O que uma linha da fato representa
Idempotência	Rodar duas vezes dá o mesmo resultado que rodar uma
OLAP / OLTP	Banco para analisar / banco para operar
Órfão	Registro que aponta para algo que não existe
Partição	Recorte (um intervalo de datas) que é apagado e recarregado
Pipeline	A cadeia que leva o dado da origem ao destino
Quarentena	Tabela onde ficam as linhas rejeitadas, com o motivo
Reconciliação	Conferir que entrada menos rejeitos é igual a saída
Staging	Área de entrada do dado bruto, em texto, antes do tratamento
Transação	Bloco "tudo ou nada": `COMMIT` ou `ROLLBACK`
Upsert	Atualiza se existe, insere se não: `ON DUPLICATE KEY UPDATE`

Apêndice C. Cola rápida

Quero	SQL
Perfilar nulos	`SUM(valor IS NULL OR valor = '')`
Achar duplicadas	`GROUP BY k HAVING COUNT(*) > 1`
Numerar repetidas	`ROW_NUMBER() OVER (PARTITION BY k ORDER BY linha_arquivo)`
Converter data	`STR_TO_DATE(txt, '%d/%m/%Y %H:%i')`
Trocar vírgula	`CAST(REPLACE(txt, ',', '.') AS DECIMAL(10,2))`
Atualizar ou inserir	`INSERT ... ON DUPLICATE KEY UPDATE col = VALUES(col)`
Recarregar partição	`DELETE ... WHERE sk_tempo BETWEEN a AND b;` e `INSERT ... SELECT`
Tudo ou nada	`START TRANSACTION; ... COMMIT;` (ou `ROLLBACK;`)
Reconciliar	`staging - rejeitos = fato`

Quero	pandas
Ler tudo como texto	`pd.read_csv(f, dtype=str)`
Nulos por coluna	`df.isna().sum()`
Duplicadas	`df.duplicated(subset=[...]).sum()`
Converter data	`pd.to_datetime(df["c"], format="%Y-%m-%d %H:%M:%S", errors="coerce")`
Corrigir valor	`df.loc[cond, "col"] = valor`
Enviar ao banco	`df.to_sql("tabela", engine, if_exists="replace", index=False)`